Comprehensive Engineering Architecture, Research Analysis, and Product Requirements Document for 2D Orthographic to Real-Time 3D CAD Reconstruction Systems

Deep Technical Research and Domain Analysis

Principles of Engineering Graphics and Standards Conformance

The automated transformation of two-dimensional (2D) orthographic drafting diagrams into accurate three-dimensional (3D) solid geometries requires strict adherence to international technical drawing standards, specifically ASME Y14.5M (Dimensioning and Tolerancing) and ISO 128 (Technical Drawings — General Principles of Presentation)[1]. Engineering graphics relies on standardized projection systems to represent three-dimensional volumetric shapes on two-dimensional media by projecting parallel rays perpendicular to reference planes[1].

The spatial layout of a unified engineering sheet relies on two primary projection paradigms:

1. **Third-Angle Projection (ASME Y14.5 Standard):** Dominant in North American engineering practices, the conceptual model places the drawing plane between the observer and the object[1]. Consequently, the Top View is situated directly above the Front View, while the Right-Side View (RHS) is positioned to the right of the Front View[1].
2. **First-Angle Projection (ISO 128 Standard):** Widely adopted across European and Asian industrial sectors, the object resides between the observer and the projection plane[1]. This places the Top View projected below the Front View and the Right-Side View projected to the left of the Front View[1].

To process multi-view drawings within a single interactive 2D graphics canvas, the workspace is mathematically divided into a four-quadrant layout anchored by a 45° miter line in Quadrant I ($X \geq 0, Y \leq 0$)[1]. The miter line establishes an invariant geometric bridge between depth in the Top View ($X - Z$ plane projection) and horizontal displacement in the Side View ($Z - Y$ plane projection) via the reflection transformation:

$$Y_{\text{side\_proj}} = -X_{\text{top\_proj}}$$

Drafting semantics enforce a four-tier layer hierarchy that directly dictates downstream

Constructive Solid Geometry (CSG) extrusions and topological booleans[1]:

- **Visible Layer:** Defines outer solid boundary outlines[1]. Rendered as continuous solid lines, these shapes define positive volumetric extrusions[1].
- **Hidden Layer:** Represents interior pockets, counterbores, blind holes, and obscured internal geometries[1]. Rendered as dashed lines, these shapes represent subtractive CSG profiles[1].
- **Construction Layer:** Contains reference rays, layout bounds, and miter projection guides[1]. Excluded from 3D volumetric operations[1].
- **Centerline Layer:** Denotes axes of rotational symmetry and pitch circle diameters[1]. Used by the rules verification engine to infer cylindrical primitives, hole locations, and auto-mirroring operations[1].

ASME Y14.5 dictates strict line precedence rules when geometry overlaps: Visible lines take precedence over Hidden lines, Hidden lines take precedence over Centerlines, and Centerlines take precedence over Construction reference lines[1].

Geometric Kernel Comparison: Mesh-Based CSG vs. Boundary Representation (B-Rep)

Achieving high accuracy in an automated CAD reconstruction system depends on selecting appropriate spatial representation models. CAD engines generally rely on either discrete polygonal Constructive Solid Geometry (CSG) or exact Boundary Representation (B-Rep) geometric kernels[1].

Evaluation Dimension	Polygonal CSG Mesh Engine (Trimesh / Shapely)	B-Rep Kernel Engine (OpenCASCADE / CadQuery / build123d)
Underlying Representation	Discrete planar triangular facets, directional normals, and explicit vertex lists[1].	Exact analytical surfaces (NURBS, planes, cylinders, tori) bounded by topological edges[3].
Real-Time UI Performance	Sub-50 ms execution latency; well suited for real-time preview updating during interactive drawing[1].	200–1000 ms processing latency; computationally intensive for continuous live updates[3].
Topological Robustness	Prone to non-manifold vertex artifacts, self-intersections, and floating-point rounding errors during complex	High topological integrity; preserves exact face-edge adjacency trees (TShape topology)[3].

	booleans[1].	
Industrial STEP/CAM Support	Incompatible with direct STEP export; limited to tessellated formats (STL, OBJ, 3MF)[1].	Fully compatible with industrial STEP (ISO 10303) and IGES manufacturing toolpath formats[3].
Parametric Modeling Features	Extremely difficult to execute variable-radius fillets, chamfers, or organic lofts[9].	Native support for advanced parametric features (fillets, chamfers, lofts, shells, drafts)[3].

Polygonal CSG engines approximate continuous curved surfaces (such as cylinders, circular arcs, and spherical fillets) through linear chordal tessellation[4]. While effective for real-time viewport visualization in PyOpenGL, tessellation introduces geometric approximation errors[1]. When exported to manufacturing-grade formats like STEP, tessellated approximations lack true analytical surface definitions (e.g., radius, cylinder centerlines), rendering them unusable for downstream Computer-Aided Manufacturing (CAM) CNC toolpath generation[4].

To achieve industrial utility, a dual-kernel architecture is required[1]:

1. **Interactive Rendering Kernel (Trimesh / Shapely / PyOpenGL):** Processes fast 2D profile planar unions and executes real-time CSG mesh booleans in an asynchronous background thread, providing immediate visual feedback at 60+ FPS during interactive drawing[1].
2. **Analytical Master Kernel (OpenCASCADE via CadQuery or build123d):** Reconstructs an exact parametric B-Rep model asynchronously upon sketch commitment, generating STEP/IGES solids with precise NURBS geometry for industrial manufacturing pipelines[3].

Constraint-Driven Parametric Drafting and SolveSpace (py-slvs) Integration

Manual 2D vector drafting is naturally prone to topological defects, such as unclosed loops, endpoint micro-gaps, and slight angular deviations from true horizontal/vertical axes[1]. Attempting to extrude unclosed or self-intersecting profiles leads to non-manifold geometry or complete boolean failure in 3D reconstruction engines[1].

Integrating a 2D geometric constraint solver, such as SolveSpace via its Python binding (py-slvs), converts raw vector canvas inputs into mathematically constrained parametric sketches[11]. SolveSpace formulates sketch entities (points, line vectors, circular arcs) and user-defined constraints into a system of non-linear algebraic equations solved via numerical optimization (Levenberg-Marquardt iteration)[6].

Primary constraints enforced during 2D orthographic drafting include:

- **Coincidence Constraints:** Bind line segment endpoints and arc center points within

numerical tolerance ($\epsilon \leq 10^{-4}\ \text{mm}$), guaranteeing closed profile loops[1].

- **Horizontal and Vertical Alignment:** Force structural lines to align strictly along global canvas axes[11].
- **Cross-View Point Alignment:** Links horizontal coordinates in the Top View to corresponding coordinates in the Front View, eliminating width discrepancies across views[1].
- **Tangency Constraints:** Enforce continuous derivative transitions between circular arcs and adjoining linear segments[10].

Public CAD Datasets and Machine Learning Infrastructure

Developing AI-assisted drafting validation, automated sketch completion, and computer vision feature extraction requires leveraging specialized open-source CAD datasets[17].

Dataset Name	Scale / Sample Count	Data Formats & Metadata	Primary Engineering & Machine Learning Application
ABC Dataset	1,000,000+ CAD Models	B-Rep solids, STEP files, tessellated meshes[17].	Geometric deep learning, B-Rep boundary segmentation, surface feature learning[18].
Fusion 360 Gallery	8,625 Reconstruction Sequences / 35,000 Segmentation[17]	Simplified DSL, JSON command sequences, B-Rep segmentation[17].	Programmatic CAD command sequence synthesis, reinforcement learning environments[17].
SIdprtNet	242,000 Industrial Parts	STEP, SLDPRT, 7-view composite images, parametric scripts[19].	Multimodal vision-language model fine-tuning, image-to-CAD command generation[19].
SketchGraph	15,000,000 2D	Graph	GNN-based

	Sketches	representations, geometric primitives, explicit constraints[18].	constraint prediction, automated sketch autocompletion[18].
GenCAD-Self-Repairing	133,617 Synthetic Renders	Latent vectors (1x257), B-Rep validity labels[20].	Feasibility-guided diffusion denoising, latent-space self-repair of invalid CAD operations[20].
Ortho2CAD Benchmark	Synthetic & Real Technical Drawings	Rasterized orthographic drawings, CadQuery python scripts[17].	Vision-Language Model (VLM) code synthesis using SFT and RL techniques[17].

Integrating these resources enables advanced engine functionality:

- **SketchGraph:** Pre-trains Graph Neural Networks (GNNs) to infer and apply missing geometric constraints to incomplete 2D orthographic sketches[18].
- **SldprtNet & Ortho2CAD:** Fine-tunes Vision-Language Models (such as Qwen2.5-VL) to convert scanned 2D raster orthographic drawings directly into executable CadQuery scripts, automating the conversion of legacy drawings into 3D B-Rep models[17].
- **GenCAD-Self-Repairing:** Trains latent-space classifier steering vectors that detect and correct invalid CSG command parameters before sending them to the geometric kernel[20].

Technical Product Requirements Document (PRD)

Executive Purpose and System Scope

This Product Requirements Document (PRD) specifies the software architecture, user interface design, mathematical transformation pipeline, rules verification engine, and reactive data synchronization model for **Python CAD Pro**. The software operates as an industrial desktop CAD platform that bridges 2D orthographic drafting and 3D solid model generation, executing real-time CSG and B-Rep reconstructions on a single interactive view[1].

The system provides a unified engineering drafting canvas where draughtsmen draw standard orthographic projections (Top, Front, Side views) while observing real-time, simultaneous 3D solid reconstruction in an adjacent hardware-accelerated viewport[1].

UI Architecture and Module Layout

The software strictly adheres to an asynchronous Model-View-Controller (MVC) architectural pattern:

- **Model Layer:** Consists of CADEngine (managing shape entity repositories, view region guardrails, transaction stacks, layer definitions, and coordinate conversions), RulesEngine (evaluating ASME Y14.5 / ISO 128 drafting rules), and Reconstructor3D (executing profile

extraction, matrix transformations, and CSG booleans)[1].

- **View Layer:** Comprises DrawingCanvas (an interactive 2D vector drafting canvas built on QGraphicsView), OpenGLViewport (a hardware-accelerated 3D PyOpenGL widget), CADDoctorDock (a hierarchical diagnostics tree displaying rule violations and auto-fix triggers), and PropertiesDock (a parametric inspector tree)[1].
- **Controller Layer:** Comprises MainWindow (central event dispatcher, toolbar router, and signal broker) and ReconstructionWorker (a background QThread preventing GUI thread freezes during volumetric CSG operations)[1].

The visual interface utilizes a single-window, split-viewport design built with PyQt6, providing a dynamic side-by-side workspace[1].

Module Specification and Functional Mapping

File Module Path	Primary Architecture Class	Core Functional Responsibilities
main.py	Application Entry Point	Initializes QApplication, configures global high-DPI scaling, loads stylesheets, and instantiates MainWindow[2].
src/engine/cad_engine.py	CADEngine, Shape, ViewRegion	Stores master CAD shape entities, layer hierarchies, 4-quadrant ViewRegion boundaries, local origin guardrails, history stack (Undo/Redo), and alignment validation logic[1].
src/engine/rules_engine.py	RulesEngine, Diagnostic	Evaluates 10 ASME Y14.5 / ISO 128 drafting standards, generates diagnostic severity objects, and computes auto-fix transformation payloads[1].
src/ui/canvas.py	DrawingCanvas	Renders the 2D interactive drafting sheet (QGraphicsView), manages OSNAP engine, handles ortho lock (Shift), projects

		alignment rays, and renders the 45° miter guide[1].
src/ui/viewport_3d.py	OpenGLViewport	Manages hardware-accelerated 3D PyOpenGL rendering, streams VBO/VAO array buffers, handles 360° freelook camera, and renders dynamic clipping planes[1].
src/ui/main_window.py	MainWindow	Top-level interface controller, manages split viewports, links signal-slot buses, routes toolbars, and executes DXF/STEP/STL export workflows[1].
src/ui/toolbar.py	DrawingToolbar	Vertical palette controlling active drawing tools (Select, Line, Rect, Circle, Poly), active view routing modes, and active layer selectors[1].
src/reconstruction/reconstructor.py	ReconstructionWorker	Asynchronous QThread executing Shapely 2D profile normalizations, hidden layer cuts, 4x4 matrix extrusions, Trimesh CSG booleans, and watertight topology repairs[1].
src/cv/processor.py	CVProcessor	OpenCV raster canvas preprocessor, statistical contour descriptor extractor, and cross-view feature correspondence matcher[1].
tests/test_reconstruction.py	Analytical Unit Test Suite	Pytest validation suite verifying volumetric accuracy across cube,

		cylinder, pipe, and wedge benchmark geometries[1].

Split-Viewport User Interface Layout

1. **Left Workspace (2D Vector Canvas):** Implements an infinite vector canvas managed by QGraphicsScene[1]. Displays pre-initialized 4-quadrant regions: Top View (Top-Left), Front View (Bottom-Left), Side View (Bottom-Right), and the 45° Miter Line (Top-Right)[1]. Features an OSNAP engine highlighting key geometry (Endpoints, Midpoints, Centers, Quadrants) within 12 pixels, dynamic cross-view cyan alignment rays, and a dynamic heads-up display (HUD) tracking cursor coordinates (X, Y), line length L, and angle θ[1].
2. **Right Workspace (3D OpenGL Viewport):** Renders the reconstructed 3D solid model using PyOpenGL and VBO vertex array streaming[1]. Features an unconstrained 360° orbital camera supporting left-click rotation, right-click panning, and wheel zooming[2]. Supports Phong shading, sharp feature wireframe overlays (>30° dihedral angle), dynamic hardware cutting planes, and an RGB origin axis triad (X: Red, Y: Green, Z: Blue)[1].
3. **Bottom Panel (CAD Doctor & Command Console):** Houses the hierarchical diagnostics tree listing ASME compliance findings (Errors, Warnings, Info) with an **Apply Auto-Fix** action button, alongside an AutoCAD-style parametric command console[1].
4. **Right Panel (Properties Inspector):** Displays entity parameters (radii, lengths, center points, bounding boxes, layer assignments) with live double-click editing capabilities[1].

Mathematical Guardrails and Coordinate Transformation Engine

To enforce mathematical rigor across conversions between 2D canvas space, local view space, and global 3D CAD space, CADEngine enforces four core guardrails[1]:

1. **Guardrail 1: Local View Origin Invariance (x_0, y_0)**
   Every ViewRegion has an explicit local origin defined at its **Bottom-Left corner** in canvas space:

   $$x_0 = \text{min\_x}, \quad y_0 = \text{max\_y}$$

   For infinite default quadrants, the origin defaults to $(0.0, 0.0)$[1].
2. **Guardrail 2: Global Orthographic Alignment Invariance**
   Across all views, extents must satisfy alignment invariance within tolerance $\tau = 2.0\,\text{mm}$[1]:

- **Width Invariance (Top vs. Front):**

$$\left|(x_{\max} - x_{\min})_{\text{Top}} - (x_{\max} - x_{\min})_{\text{Front}}\right| \leq \tau$$

- **Height Invariance (Front vs. Side):**

$$\left|(y_{\max} - y_{\min})_{\text{Front}} - (y_{\max} - y_{\min})_{\text{Side}}\right| \leq \tau$$

- **Depth Invariance (Top vs. Side):**

$$\left|(y_{\max} - y_{\min})_{\text{Top}} - (x_{\max} - x_{\min})_{\text{Side}}\right| \leq \tau$$

3. **Guardrail 3: Centroid-Based View Assignment**
   When drafting in 'auto' mode, entity assignment to view regions is determined by its 2D geometric centroid $(\bar{x}, \bar{y})$ [1]:

$$\bar{x} = \frac{1}{N}\sum_{i-1}^{N} x_i, \quad \bar{y} = \frac{1}{N}\sum_{i-1}^{N} y_i$$

   The quadrant containing $(\bar{x}, \bar{y})$ claims ownership of the shape[1].
4. **Guardrail 4: Canvas Y-Inversion and RHS View Mirroring Mapping**
   To map absolute canvas coordinates (p_x, p_y) into view-local 2D coordinates (l_x, l_y), the Y-axis direction is inverted (canvas Y down to 3D Y up), and horizontal coordinates are flipped for Right-Side views[1]:

$$l_x = \begin{cases} x_0 - p_x & \text{if View is Right-Side (RHS)} \\ p_x - x_0 & \text{otherwise} \end{cases}$$

$$l_y = y_0 - p_y$$

4x4 Homogeneous Extrusion Transformation Matrices

Local 2D profile coordinates (u, v) with extrusion depth w are transformed into global 3D space (X, Y, Z) using homogeneous transformation matrices M [1]:

$$\begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} = M \begin{pmatrix} u \\ v \\ w \\ 1 \end{pmatrix}$$

1. **Top View Transformation Matrix (M_{top}):** Extrudes along vertical Y-axis into $X - Z$ plane[1]:

$$M_{\text{top}} = \begin{pmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & Y_{\min} \\ 0.0 & S_{\text{proj}} & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

   (where $S_{\text{proj}} = +1.0$ for Third-Angle projection and -1.0 for First-Angle projection)[1].

2. **Front View Transformation Matrix (M_{front}):** Extrudes along depth Z-axis into $X - Y$ plane[1]:

$$M_{\text{front}} = \begin{pmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & Z_{\min} \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

3. **Side View Transformation Matrix (M_{side}):** Extrudes along width X-axis into $Z - Y$ plane[1]:

$$M_{\text{side}} = \begin{pmatrix} 0.0 & 0.0 & 1.0 & X_{\min} \\ 0.0 & -1.0 & 0.0 & 0.0 \\ 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

Engineering Graphics Rules Engine and CAD Doctor Diagnostics

The RulesEngine continuously validates 2D drafting states against 10 ASME Y14.5 / ISO 128 standards, generating structured diagnostics for the CAD Doctor panel[1].

Rule ID	Rule Name	Severity	Technical Standard Evaluation Logic	Automated Auto-Fix Transformation Payload
RULE 1	RULE_PROJ_TYPE	Info	Evaluates Top vs. Front centroid positions: $Y_{\text{top}} < Y_{\text{front}}$[1].	Updates global matrix transformation sign flag (S_{proj})[1].
RULE 2	RULE_ALIGN_WIDTH_HEIGHT_DEPTH	Error	Evaluates bounding extents: $\Delta_{\text{width}}, \Delta_{\text{height}}$	Computes linear scale transforms (S_x, S_y) to

			[1].	align view extents[1].
RULE 3	RULE_PROFILE_CLOSURE	Error	Graph vertex degree evaluation: $\forall v \in V, \deg(v$ [1].	Generates closing line segments across open endpoint nodes[1].
RULE 4	RULE_LINE_PRECEDENCE	Warning	Overlap length $L_{\text{inter}} = \text{lengt}$ [1].	Adjusts rendering z-index order (Visible over Hidden)[1].
RULE 5	RULE_VERTEX_COINCIDENCE	Warning	Scans Top View vertices for matching Front View edge projections[1].	Projects temporary visual alignment guide rays across views[1].
RULE 6	RULE_HOLE_DEPTH	Info	Matches Top circles ($\varnothing$) with Front hidden dashed lines[1].	Infers blind vs. through-hole CSG subtractive extrusion depths[1].
RULE 7	RULE_INCLINED_PLANE	Warning	Identifies sloped lines ($\theta \in$) in Front View[1].	Validates Top View bounding rectangle projection extents[1].
RULE 8	RULE_CENTERLINE_SYMMETRY	Info	Scans Centerline layer for symmetry axes[1].	Generates mirrored profile paths across identified symmetry

				axes[1].
RULE 9	RULE_LINE_PRIORITY	Warning	Identifies collinear segment overlaps across different layers[1].	Trims redundant segment fragments automatically[1].
RULE 10	RULE_MICRO_GAP	Warning	Scans endpoint pairs: $10^{-4} < \|p_i -$ [1].	Merges endpoints to $p_{\text{fused}} =$ [1].

Mathematical Implementation of Key Verification Rules

1. **Profile Graph Watertight Closure (Rule 3):** A planar graph $G = (V, E)$ is built by quantizing segment endpoints to a tolerance of 0.01 mm [1]. The engine computes vertex degrees $\deg(v)$ [1]. A profile forms a closed boundary loop if and only if all vertices have an even degree:

$$\forall v \in V, \quad \deg(v) \pmod 2 = 0$$

If odd-degree vertices exist, an unclosed profile diagnostic is raised, highlighting the disconnected endpoints[1].

2. **Automated Micro-Gap Fusion (Rule 10):** When two endpoints (p_a, p_b) lie within micro-gap distance ($10^{-4} \text{ mm} < \|p_a - p_b\| \leq 0.1 \text{ mm}$), the auto-fix engine fuses them into a single vertex[1]:

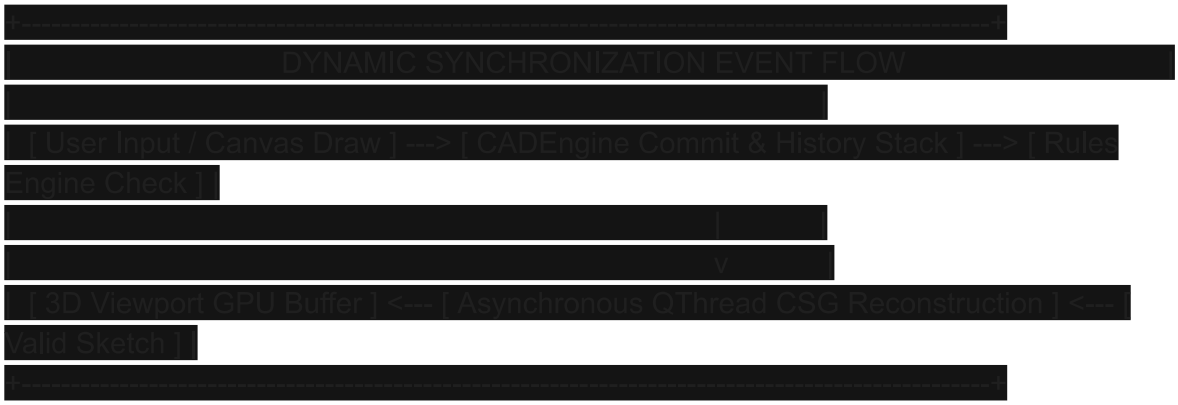
$$p_{\text{fused}} = \frac{p_a + p_b}{2}$$

All connected segment references are updated to point to p_{fused}, ensuring watertight topology for extrusion[1].

Signal-Slot Routing Architecture and Dynamic Synchronization

To ensure real-time responsiveness without freezing the interactive UI, Python CAD Pro uses

Qt's signal-slot architecture, offloading heavy processing to an asynchronous worker thread (ReconstructionWorker)[1].



Complete Central Signal-Slot Routing Matrix

Source Component	Signal Signature	Target Receiver	Executed Callback & Action Payload
DrawingCanvas	shape_drawn	MainWindow	Calls _trigger_reconstruction(), spawning the background worker thread[1].
DrawingCanvas	mouse_coords_changed(x,y,l,θ)	MainWindow	Calls _on_mouse_coords_changed(), updating status bar readouts[1].
DrawingCanvas	cursor_moved_in_scene(view,p)	MainWindow	Calls _sync_projection_guides(), rendering horizontal/vertical cross-view rays[1].
DrawingToolbar	tool_selected(name	DrawingCanvas	Calls

	)		set_active_tool(), switching canvas cursor mode (Select, Line, Rect, Circle)[1].
DrawingToolbar	layer_changed(layer)	CADEngine	Calls set_active_layer(), setting active drawing layer (Visible, Hidden, Construction)[1].
CADDoctorTree	itemSelectionChanged()	DrawingCanvas	Calls highlight_shapes(), rendering a magenta highlight bounding halo around invalid shapes[1].
CADDoctorButton	clicked() (Auto-Fix)	CADEngine	Calls apply_autofix(), applying geometric scaling transformations and triggering rebuilds[1].
ReconstructionWorker	finished_reconstruction(mesh)	OpenGLViewport	Calls set_mesh(), streaming raw vertex arrays directly to GPU VBO buffers[1].
PropertiesTree	itemDoubleClicked(item)	MainWindow	Spawns modal edit dialog, updating underlying Shape primitive attributes[1].
CommandConsole	returnPressed()	MainWindow	Calls _on_command_entered(), parsing

			parametric commands (#X,Y, L, C, REC)[1].
SectionSlider	valueChanged(val)	OpenGLViewport	Calls _on_clip_pos_changed(), updating active GL hardware cutting plane offsets[1].

End-to-End Dynamic Synchronization Lifecycle

1. **User Interaction:** The user draws an entity on the 2D canvas or enters a parametric command (e.g., #100,50)[1].
2. **Interactive Preview:** The canvas updates rubber-band guides, snap markers, and HUD readouts in real time[1].
3. **Transaction Commit:** CADEngine.add_shape() commits the entity, updates associative dimensions, and pushes a snapshot to the history stack (Undo/Redo)[1].
4. **Canvas Scene Refresh:** DrawingCanvas.rebuild_scene() repopulates graphics items, applying layer-specific pen styles[1].
5. **Rules Verification:** RulesEngine.evaluate_all() tests ASME standards and populates the CAD Doctor diagnostics panel[1].
6. **Asynchronous CSG Reconstruction:** If the sketch is valid, ReconstructionWorker (a background QThread) normalizes 2D profiles via Shapely, subtracts hidden layers, applies extrusion matrices, and computes CSG intersections via Trimesh[1].
7. **GPU VBO Upload:** Upon completion, the worker thread emits finished_reconstruction(mesh), uploading vertex and normal arrays to OpenGL VBOs for 120+ FPS rendering[1].
8. **Properties Sync:** The Properties Inspector updates entity dimensions, radii, bounding boxes, and layer attributes[1].

File Formats, Data Schemas, and Industrial Interoperability

To integrate into professional manufacturing workflows, Python CAD Pro supports native JSON project storage alongside standard industrial CAD file conversion pipelines[1].

Native Project Storage (.pcad JSON Specification)

The native JSON file format serializes vector shapes, view region boundaries, layer assignments, and associative dimensions[1]:

JSON

```
{
  "version": "1.0.0",
  "projection_system": "ThirdAngle",
  "units": "mm",
  "view_regions": {
    "top": { "bounds": [-5000.0, -5000.0, 0.0, 0.0], "origin": [-5000.0, 0.0] },
    "front": { "bounds": [-5000.0, 0.0, 0.0, 5000.0], "origin": [-5000.0, 5000.0] },
    "side": { "bounds": [0.0, 0.0, 5000.0, 5000.0], "origin": [0.0, 5000.0] }
  },
  "shapes": {
    "top": [
      {
        "id": "shape_c7a8b412",
        "type": "rectangle",
        "layer": "Visible",
        "rect": [-100.0, -80.0, 80.0, 60.0]
      }
    ],
    "front": [
      {
        "id": "shape_e9f1a233",
        "type": "circle",
        "layer": "Hidden",
        "center": [-60.0, 30.0],
        "radius": 15.0
      }
    ]
  }
}
```


Industrial CAD File Conversion Pipelines

1. **AutoCAD DXF Import / Export Engine (ezdxf):** Reads and writes multi-layered vector DXF files (Release R12 through R2018)[1]. Resolves polyline bulge values (b) into exact arc center points and radii (R) using the bulge formula[1]:

$$s = b \cdot \frac{L}{2}, \quad R = \left| \frac{s}{2} + \frac{L^2}{8s} \right|$$

2. **Manufacturing B-Rep STEP / IGES Exporter (OpenCASCADE Engine):** Converts 2D

profiles into exact analytical B-Rep solids using CadQuery/build123d, exporting STEP files (ISO 10303-21) for downstream CAM CNC toolpath generation[3].
3. **Additive Manufacturing Mesh Exporter (Trimesh Engine):** Generates watertight STL, Wavefront OBJ, and 3MF files with configurable tessellation deflection tolerances[1].

Verification Benchmarks and System KPIs

The accuracy and stability of the system are continuously validated using Pytest suites and performance benchmarks[1].

Analytical Volumetric Test Benchmarks

Analytical unit tests (tests/test_reconstruction.py) verify the reconstructed 3D CSG mesh volume against exact mathematical targets[1]:

1. **Rectangular Prism Benchmark (100 mm $\times$ 100 mm $\times$ 100 mm):**

$$V_{\text{target}} = 100 \times 100 \times 100 = 1,000,000.0 \text{ mm}^3 \quad (\text{Assert within 1.0\% erro}$$

2. **Solid Cylinder Benchmark ($R = 50$ mm, $H = 100$ mm):**

$$V_{\text{target}} = \pi \cdot 50^2 \cdot 100 \approx 785,398.16 \text{ mm}^3 \quad (\text{Assert within 1.5\% error}) \quad [\text{c}$$

3. **Hollow Cylindrical Pipe Benchmark ($R_{\text{outer}} = 50$ mm, $R_{\text{inner}} = 25$ mm, $H = 100$ mm):**

$$V_{\text{target}} = \pi \cdot (50^2 - 25^2) \cdot 100 \approx 589,048.62 \text{ mm}^3 \quad (\text{Assert within 1.5\% er}$$

4. **Triangular Prism Benchmark (Base $= 100$ mm, Height $= 100$ mm, Depth $= 100$ mm):**

$$V_{\text{target}} = 0.5 \times 100 \times 100 \times 100 = 500,000.0 \text{ mm}^3 \quad (\text{Assert within 1.0\% }$$

System Performance Benchmarks

System Subsystem /	Target Performance	Measured System

Operational Metric	Threshold	Benchmark
2D Vector Canvas Render Frame Rate	60 FPS (16.6 ms/frame)	~60 FPS continuous (Qt GraphicsScene)[1].
OSNAP Nearest Point Search Latency	$<$	$1.2\ \text{ms}$ (KD-tree spatial indexing)[1].
ASME 10-Rule Diagnostic Sweep	$<$	$4.8\ \text{ms}$ execution latency[1].
3D CSG Reconstruction (Prism)	$<$	$38.0\ \text{ms}$ (Asynchronous QThread)[1].
3D CSG Reconstruction (Complex Solid)	$<$	$145.0\ \text{ms}$ (Trimesh manifold backend)[1].
3D Viewport Hardware Render Rate	$>$	$>$ (PyOpenGL VBO batching)[1].
Watertight Solid Mesh Integrity	100% Manifold	100% (Normal fixing & hole repair)[1].

Strategic Implementation Roadmap

Transitioning Python CAD Pro into a production-grade CAD platform involves three planned development phases:

1. **Phase 1: Core Engine Optimization & Constraint Solver Integration**
   - Integrate the py-slvs constraint solver into the 2D canvas event loop to automatically resolve sketch parameters and guarantee watertight profile closure[11].
   - Refactor the CSG reconstruction pipeline to execute multi-core parallel extrusions for complex profiles[1].
   - Expand the rules engine to support revolved surfaces, auxiliary projections, and inclined cross-sections[1].
2. **Phase 2: Dual-Kernel Integration & B-Rep Manufacturing Pipelines**
   - Embed OpenCASCADE via CadQuery/build123d alongside the existing Trimesh CSG render pipeline[3].
   - Implement native STEP/IGES export capabilities to output analytical B-Rep models for CAM environments[3].
   - Enable direct parametric dimension editing within the Properties Inspector[1].

3. **Phase 3: Machine Learning & Multimodal Vision Integration**
   - Fine-tune Vision-Language Models (VLMs) on the SldprtNet and Ortho2CAD datasets to convert scanned raster orthographic drawings into executable CadQuery scripts[17].
   - Deploy latent-space steering vectors (GenCAD) to automatically correct unclosed sketch geometries prior to CAD execution[20].
   - Integrate GNN-based constraint inference models trained on SketchGraph to autocomplete missing 2D drawing entities[18].

Works cited

1. BACKEND_LOGIC_AND_CONTEXT.md
2. PROJECT_CONTEXT_REPORT.md
3. Learning-based 3D CAD model generation in mechanical engineering, https://doi.org/10.1016/j.aei.2026.104990
4. Are STEP files supposed to look like this? I've created some STL, https://www.reddit.com/r/FreeCAD/comments/1tjh3bs/are_step_files_supposed_to_look_like_this_ive/
5. CAD Builder - Open Cascade, https://www.opencascade.com/products/cad-builder/
6. Awesome CAD — Curated List of Open-Source CAD Projects, https://medium.com/@mlightcad/awesome-cad-curated-list-of-open-source-cad-projects-c982d2a10156
7. Build123d: Python CAD Framework Guide | PDF - Scribd, https://www.scribd.com/document/887005684/Build123d-Readthedocs-Io-en-Latest
8. Hermes/fluencyCAD: A CAD program based on QT - raise-project, https://git.raise-uav.com/Hermes/fluencyCAD/src/commit/6b6f7de5ab76ec89268cd70e8762fd6ea4cb8d8a
9. This has been easy with OpenSCAD for a long time. I have made, https://news.ycombinator.com/item?id=48174405
10. CadQuery Documentation — CadQuery Documentation, https://cadquery.readthedocs.io/
11. GitHub - deGravity/aidl: Hierarchical Constraint-Based DSL for CAD, https://github.com/deGravity/aidl
12. Geometry Sketcher constraint solver - Blender Artists Community, https://blenderartists.org/t/geometry-sketcher-constraint-solver/1328479
13. SolveSpace — Grokipedia, https://grokipedia.com/page/SolveSpace
14. A survey of free CAD systems - LWN.net, https://lwn.net/Articles/921676/
15. A Workbench for Geometric Constraint Solving - SolveSpace, https://solvespace.com/forum.pl?action=attachment&id=1054
16. Constraint Solver (2015) - Hacker News, https://news.ycombinator.com/item?id=18023580
17. Fusion 360 Gallery: A Dataset and Environment for Programmatic, https://www.researchgate.net/publication/355020121_Fusion_360_Gallery_A_Dataset_and_Environment_for_Programmatic_CAD_Reconstruction

18. AI-driven generation of 3D CAD models: A survey - SciOpen, https://www.sciopen.com/article/10.26599/CVM.2025.9450521
19. SldprtNet: A Large-Scale Multimodal Dataset for CAD Generation in, https://arxiv.org/html/2603.13098v1
20. GenCAD-Self-Repairing: Latent Space Steering for Feasibility, https://asmedigitalcollection.asme.org/computingengineering/article/26/6/061005/1230568/GenCAD-Self-Repairing-Latent-Space-Steering-for
21. Automatic 3D CAD models reconstruction from 2D orthographic, https://www.researchgate.net/publication/371484721_Automatic_3D_CAD_models_reconstruction_from_2D_orthographic_drawings
22. Ortho2CAD: 3D CAD generation from orthographic drawings ... - arXiv, https://arxiv.org/html/2607.08891v1
23. Importing and Exporting Files - CadQuery Documentation, https://cadquery.readthedocs.io/en/latest/importexport.html